

Week 3 - Day 13b Supplementary: Graphs

We will continue our exploration of graphs in this supplementary.

```
# Install the libraries
%pip install networkx matplotlib

import networkx as nx

def draw(graph):
    nx.draw(nx.DiGraph(graph), with_labels=True, node_size=1000,
            node_color='#cfc8f4', arrowsize=40)
```

Exercise 1: Adjacency Matrix $[[[]]]$

If the nodes of a graph are the numbers from 0 to $n-1$, we can also represent it as an $n \times n$ matrix called the **adjacency matrix**.

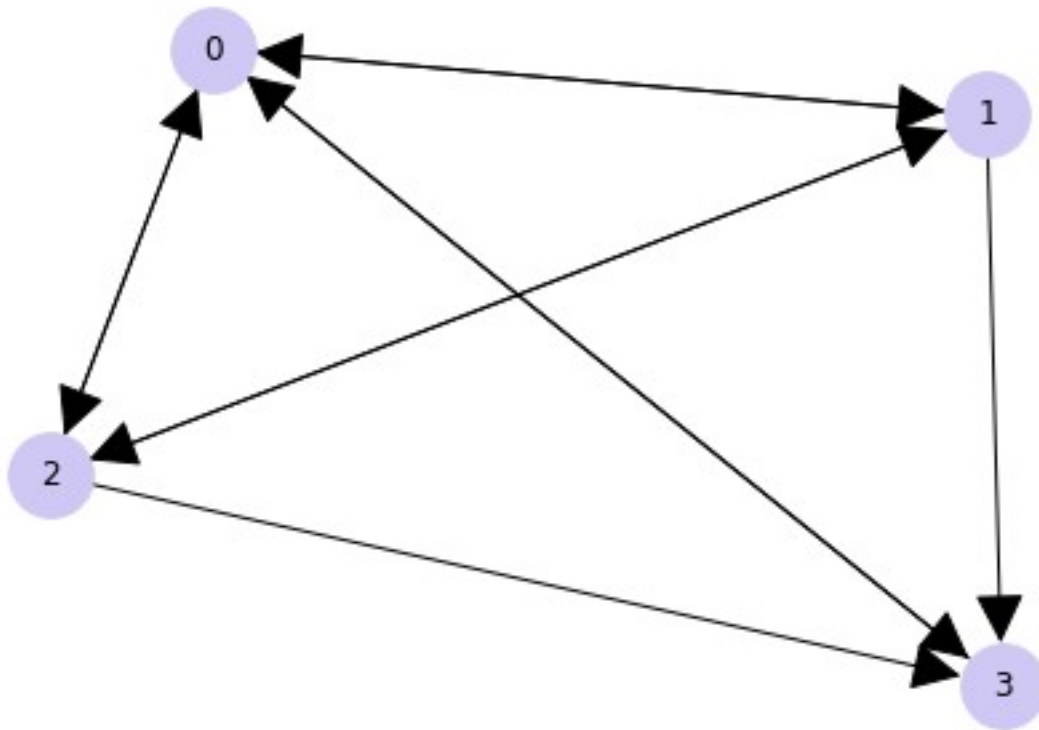
$$A = \begin{pmatrix} a_{00} & a_{01} & \cdots & a_{0(n-1)} \\ a_{10} & a_{11} & \cdots & a_{1(n-1)} \\ \vdots & \vdots & \ddots & \vdots \\ a_{(n-1)0} & a_{(n-1)1} & \cdots & a_{(n-1)(n-1)} \end{pmatrix}$$

Its entries a_{ij} are **1** if there is an edge from i to j , and **0** otherwise. For example, for the graph below

the adjacency matrix is

$$A = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

1.1 What is the **adjacency matrix** of the following graph?



1.2 Write a function `zeromat(n)` that returns an $n \times n$ matrix whose entries are all 0.

Hint: Use a nested list in Python to represent a matrix

```

#TODO: Your code here
def zeromat(n):
    pass

# Use this cell to test your code
assert zeromat(1) == [[0]]
assert zeromat(2) == [[0,0],[0,0]]
assert zeromat(3) == [[0,0,0],[0,0,0],[0,0,0]]

print("All tests passed. ☐")

```

1.3 Write a function `number_nodes(lst)` that takes a **list of edges** and returns the number of nodes in the graph.

```

# TODO: Your code here
def number_nodes(edge_list):
    pass

# # Use this cell to test your solution
assert number_nodes([[0, 1], [1, 2]]) == 3
assert number_nodes([[0, 0]]) == 1
assert number_nodes([[0, 1], [1, 2], [3, 4]]) == 5

```

```
print("All tests passed. ☐")
```

1.4 Write a function `to_adj_matrix(lst)` that takes a **list of edges** and turns it into an **adjacency matrix**.

Hint: You can use your function `zeromat` after computing how many nodes are in the graph.

```
# TODO: Your code here
def to_adj_matrix(edge_list):
    pass

# Use this cell to test your solution
assert to_adj_matrix([[0,1], [0,2], [1,0], [1,2], [2,0]]) == [[0, 1, 1], [1, 0, 1], [1, 0, 0]]
assert to_adj_matrix([[0,1], [0,2], [0,3], [1,0], [1,2], [1,3], [2,0], [2,1], [2,3], [3,0]]) == [[0, 1, 1, 1], [1, 0, 1, 1], [1, 1, 0, 1], [1, 0, 0, 0]]

print("All tests passed. ☐")
```

Exercise 2: Counting Walks [☐☐☐]

A walk of length n is a sequence of $n+1$ vertices, $v_0, v_1, \dots, v_n$, such that (v_i, v_{i+1}) is an edge for all $i \in \{0, 1, \dots, n-1\}$. For example, in the below graph, the sequence $(0, 1, 0, 2)$ is a walk of length 4.

The sequence $(0, 1, 2)$ is **not** a walk because there is no edge from 1 to 2. In the above graph, how many walks of **length 2** exist from vertex...

1. 0 to 0?
2. 0 to 1?
3. 0 to 2?
4. 1 to 0?
5. 1 to 1?
6. 1 to 2?
7. 2 to 0?
8. 2 to 1?
9. 2 to 2?

Exercise 3: Matrix Multiplication! [☐☐☐☐]

We will now discover an elegant way to compute the number of such paths. If A and B are $n \times n$ matrices,

$$\mathbf{A} = \begin{pmatrix} a_{00} & a_{01} & \cdots & a_{0(n-1)} \\ a_{10} & a_{11} & \cdots & a_{1(n-1)} \\ \vdots & \vdots & \ddots & \vdots \\ a_{(n-1)0} & a_{(n-1)1} & \cdots & a_{(n-1)(n-1)} \end{pmatrix}$$

$$\mathbf{B} = \begin{pmatrix} b_{00} & b_{01} & \cdots & b_{0(n-1)} \\ b_{10} & b_{11} & \cdots & b_{1(n-1)} \\ \vdots & \vdots & \ddots & \vdots \\ b_{(n-1)0} & b_{(n-1)1} & \cdots & b_{(n-1)(n-1)} \end{pmatrix}$$

the **matrix product** AB is the matrix

$$C = \begin{pmatrix} c_{00} & c_{01} & \cdots & c_{0(n-1)} \\ c_{10} & c_{11} & \cdots & c_{1(n-1)} \\ \vdots & \vdots & \ddots & \vdots \\ c_{(n-1)0} & c_{(n-1)1} & \cdots & c_{(n-1)(n-1)} \end{pmatrix}$$

whose entries are

$$c_{ij} = a_{i0}b_{0j} + a_{i1}b_{1j} + \cdots + a_{i(n-1)}b_{(n-1)j}$$

For example, for a 2×2 matrix A and B , we can compute the product as follows:

$$A = \begin{pmatrix} 1 & 2 \\ 0 & 4 \end{pmatrix}$$

$$B = \begin{pmatrix} 5 & 3 \\ -1 & 0 \end{pmatrix}$$

$$AB = \begin{pmatrix} 1 & 2 \\ 0 & 4 \end{pmatrix} \begin{pmatrix} 5 & 3 \\ -1 & 0 \end{pmatrix} = \begin{pmatrix} 1 \cdot 5 + 2 \cdot (-1) & 1 \cdot 3 + 2 \cdot 0 \\ 0 \cdot 5 + 4 \cdot (-1) & 0 \cdot 3 + 4 \cdot 0 \end{pmatrix} = \begin{pmatrix} 3 & 3 \\ -4 & 0 \end{pmatrix}$$

3.1 Consider the adjacency matrix of our graph from before:

$$A = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$

Compute $A^2 = A \cdot A$ by on paper.

3.2 Compare the entries of A^2 with the number of walks of length 2 you counted in **Exercise 2.1**. What do you notice?

3.3 Write a function `matmul(A, B)` that takes two matrices and returns their **matrix product**.

Hint: The function `zeromat` from earlier might be useful.

```
# TODO: Your code here
def matmul(a, b):
    pass

# Use this cell to test your code
assert matmul([[1, 2], [3, 4]], [[5, 6], [7, 8]]) == [[19, 22], [43,
```

```
50]]
assert matmul([[1, 0, 1], [2, 1, 1], [0, 1, 1]], [[1, 2, 0], [2, 3, 1], [1, 1, 0]]) == [[2, 3, 0], [5, 8, 1], [3, 4, 1]]
assert matmul([[1, 2, 3], [4, 5, 6]], [[7, 8], [9, 10], [11, 12]]) == [[58, 64], [139, 154]]
assert matmul([[1, 2], [3, 4]], [[0, 0], [0, 0]]) == [[0, 0], [0, 0]]

print("All test passed. ☐")
```

3.4 Use your code to compute the number of paths of length 2 for our graph from Exercise 2.

```
# TODO: Your code here
edge_list = [[0, 1], [0, 2], [1, 0], [1, 2], [2, 0]]
```

3.5 For our graph from earlier with adjacency matrix,

$$A = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$

compute $A^3 = A \cdot A \cdot A$ using your `matmul` function.

```
# TODO: Your code here
```

Compute the entries of S with the number of walks of length 3 for our graph from Exercise 2.2! What do you notice?

3.6 We've seen in the previous exercise that it is a bit awkward to compute powers of a matrix using `matmul`. Write a new function `matpow(A, n)` that computes A^n , the matrix A to the n -th power, using `matmul`.

```
# TODO: Your code here
def matpow(A, n):
    pass

# Tests for your code
a = [[0, 1, 1], [1, 0, 1], [1, 0, 0]]
assert matpow(a, 2) == [[2, 0, 1], [1, 1, 1], [0, 1, 1]]
assert matpow(a, 3) == [[1, 2, 2], [2, 1, 2], [2, 0, 1]]
assert matpow(a, 4) == [[4, 1, 3], [3, 2, 3], [1, 2, 2]]
b = [[1, 2], [0, 1]]
assert matpow(b, 5) == [[1, 10], [0, 1]]

print("All tests passed. ☐")
```

3.7 Think about we can read the number of paths of length n from i to j from the (i, j) entry of A^n .

Hint: If you know the number of paths of length $n - 1$ from x to j for all nodes j , how can you find out the number of paths of length n from x to y ?

Exercise 4: Equal Sum of Vertices[]

Consider a graph with N , where $N \geq 3$ vertices numbered from 1 to N and without edges. You need to make it connected (every vertex should be reachable from each other) in a such way that the number of the adjacent vertices is equal for each vertex.

Below you can see such graph for $N=3$.

```
# TODO: Your code here
def solve(n):
    pass

# Use this cell to test your code
def check(graph):
    def is_connected(graph):
        visited = set()
        def visit(node):
            visited.add(node)
            for neighbor in graph[node]:
                if neighbor not in visited:
                    visit(neighbor)
        visit(list(graph.keys())[0])
        return len(visited) == len(graph)

    assert is_connected(graph)
    assert all(len(graph[i]) == len(graph[j]) for i in graph for j in
graph)
    return True

assert check(solve(3))
assert check(solve(4))
assert check(solve(5))
assert check(solve(6))

print("All tests passed. [ ]")
```

Congratulations []